

Session 4: Building the Autonomous AI Agent (RAG Architecture)

Chapter 6: Conversational Agent and RAG Architecture (Chat with Your Documents).....	1
1. Flow Isolation: Creating a New Trigger.....	1
2. Adding the "Brain" (AI Agent Node).....	2
3. Agent Equipment (Model, Memory and RAG Tool).....	3
4. Agent Translator (Embeddings).....	5
5. The Moment of Truth: The Conversation with Your Data!.....	6

Chapter 6: Conversational Agent and RAG Architecture (Chat with Your Documents)

 **In this final chapter, we give a voice to our assistant.** We have the files downloaded and transformed into vectors in Pinecone. Now we will build a Chat interface in n8n, where an AI Agent will read our questions, search for the answer in Pinecone, and formulate a human and technical response for us.

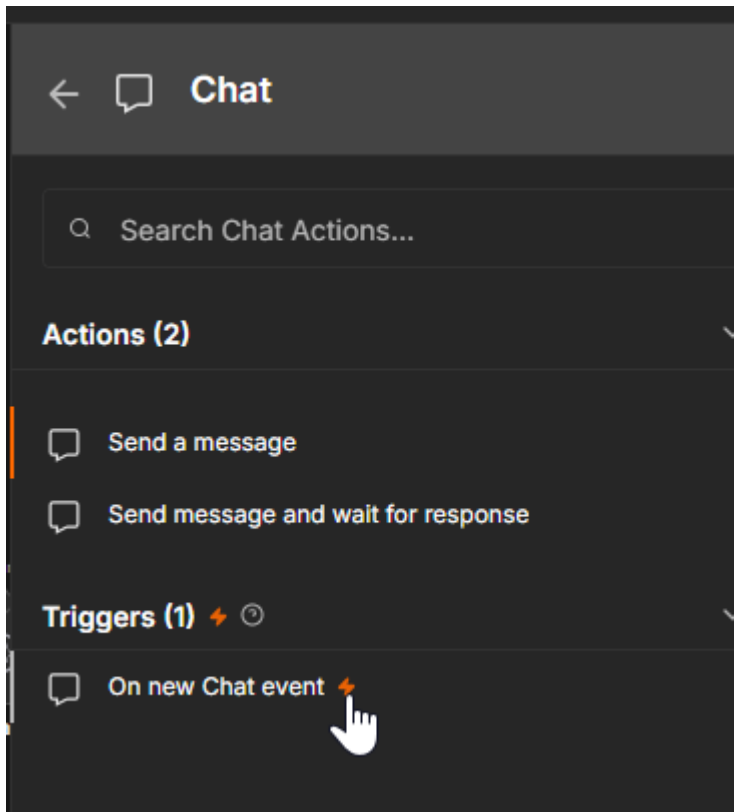
1. Flow Isolation: Creating a New Trigger

Until now, our stream was triggered by an event in Google Drive. But now we want the action to start when We write a message to the assistant.

To keep the canvas clean and not download files every time we talk to the AI, we'll add a completely separate starting point on the same canvas (or you can create a new Workflow, if you prefer).

- Double-click anywhere on an empty space on your canvas to add a new node, away from the old flow.

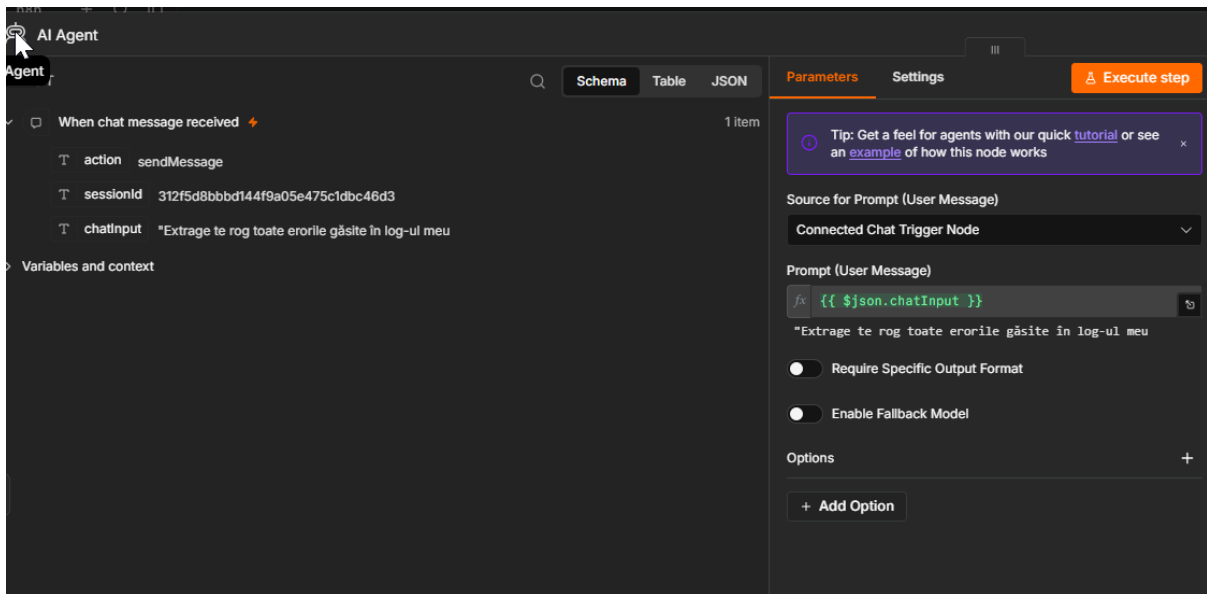
- Search in the search bar directly for the option **Chat** (the first one in the list, with the simple message icon) and click on it.
- A sub-menu will open. **Skip the Actions section !** Go straight to the section **Triggers** (marked with a small orange lightning bolt) and open it. Click on the option there called **On new Chat event** The node that will appear on the board is designed to open an interactive chat window directly in the n8n interface.



2. Adding the "Brain" (AI Agent Node)

The agent is the "manager" of the operation. He decides whether he can answer you from memory or if he needs to use a tool to search the documents.

- Pull a thread from your new Chat node and search for the node **AI Agent** (you can find it in the section *Advanced AI*).



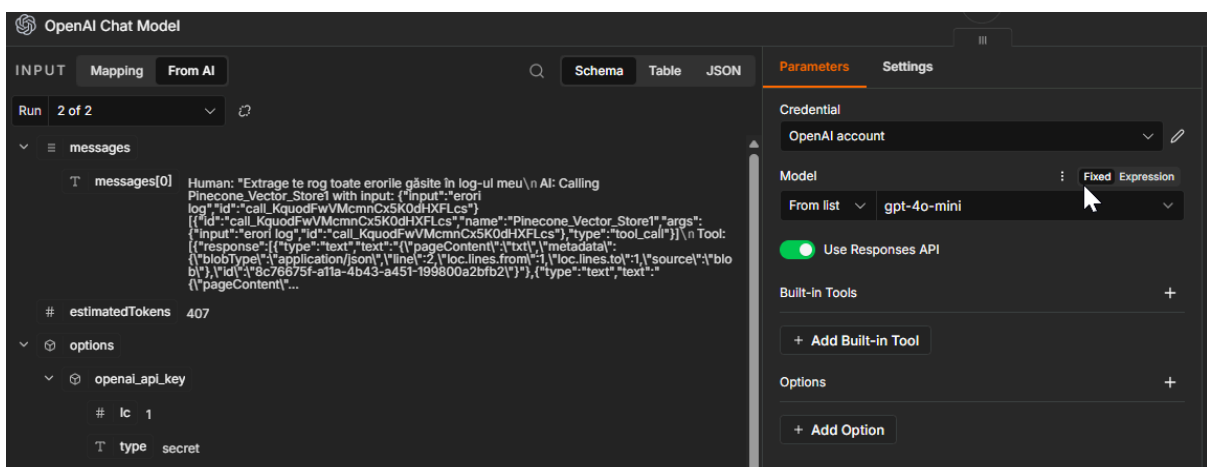
- Once added, you will see that this node has several lower connection ports, each with a specific role to build our digital "little man".

3. Agent Equipment (Model, Memory and RAG Tool)

For an agent to function properly, it needs a personality, a memory, and a way to search our database. We'll add these one by one using its lower ports:

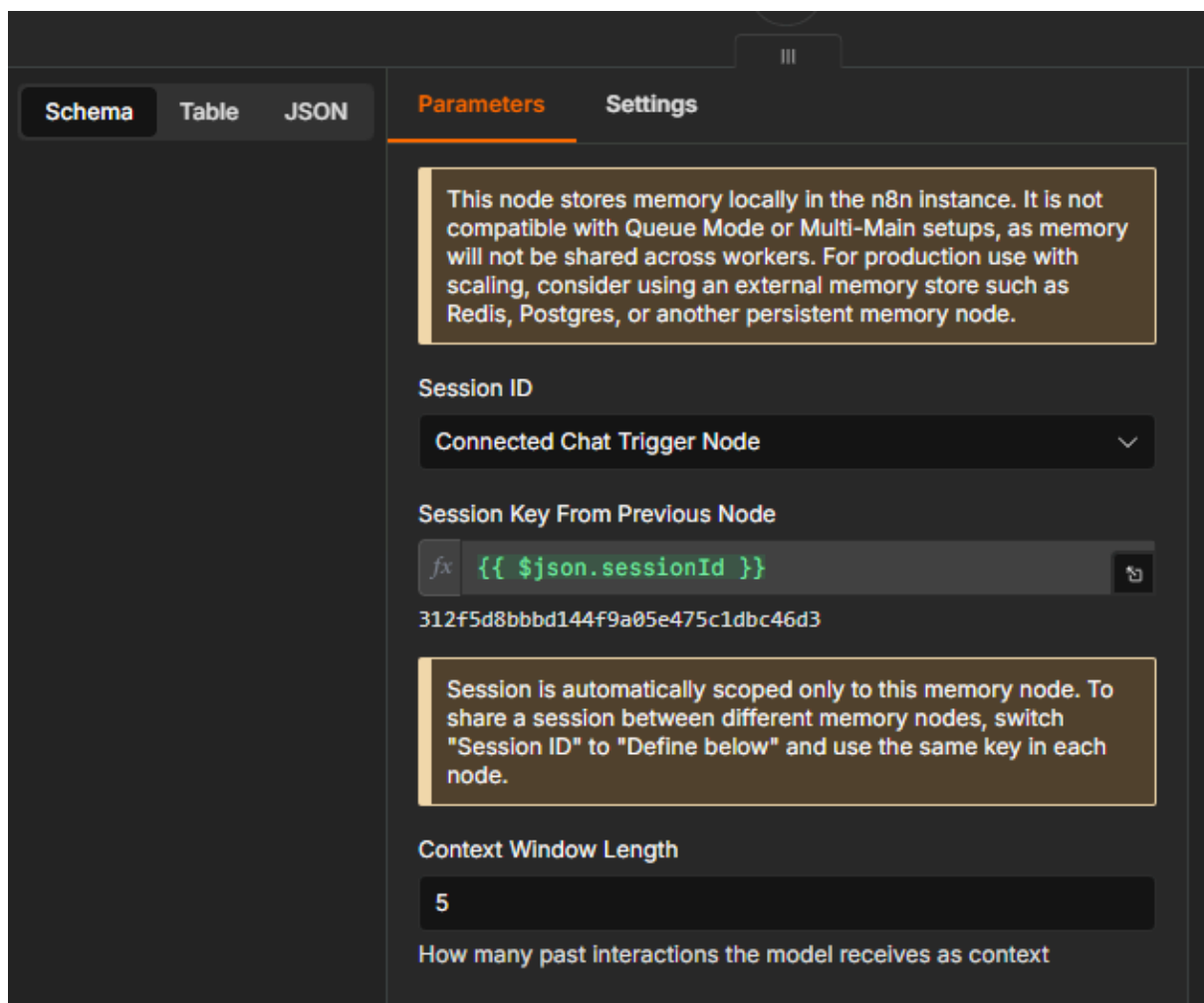
A. The Model (Mouth and Personality)

- Click on the lower left port of the node *AI Agent*, called **Model**.
- Choose **OpenAI Chat Model**.
- Select the same credentials (API key) that you created in the previous chapter.
- In the Model section, select gpt-4o-mini (it's extremely fast, smart, and costs fractions of a cent).



B. Memory (Conversation Context)

- Press the bottom center port of the node *AI Agent*, called **Memory**.
- Choose option **Simple Memory** (you can find it under the category *For beginners*).
- *Why are we doing this?* Without memory, if you ask him "What is a 404 error?", he will answer you, but if you ask him the next question "And how do I fix it?", he will completely forget what you were referring to in the first sentence! *Simple Memory* acts as a buffer, retaining the history of the current discussion for the duration of the session, directly in n8n's RAM.



The screenshot shows the configuration interface for the 'Simple Memory' node in n8n. The interface is divided into two tabs: 'Parameters' (selected) and 'Settings'. On the left, there are tabs for 'Schema', 'Table', and 'JSON'. The main configuration area includes a warning box at the top stating that the node stores memory locally and is not compatible with Queue Mode or Multi-Main setups. Below this, the 'Session ID' is set to 'Connected Chat Trigger Node'. The 'Session Key From Previous Node' is configured with a JavaScript expression: `{{ $json.sessionId }}`. A generated session key is displayed below: `312f5d8bbbd144f9a05e475c1dbc46d3`. Another warning box explains that the session is automatically scoped to this memory node and can be shared by switching the 'Session ID' to 'Define below'. The 'Context Window Length' is set to 5, with a description: 'How many past interactions the model receives as context'.

C. Search Tool (Pinecone Vector Store)

This is where the magic happens, connecting this chat with our work in Chapter 5. The n8n platform allows us to link our database directly as a "tool" that the AI uses itself.

- Click on the lower right port of the node *AI Agent*, called **Tool** (or on the + sign below it).
- In the menu that appears, ignore the categories above and look for the node **Pinecone**

Vector Store.

- Once you've attached it to the board, click on it to open its settings. On the parameter **Operation Mode** , make sure it is selected **Retrieve Documents (As Tool for AI Agent)** This tells the node that its role is not to write data (as in Chapter 5), but to read it.
- At the field **Description** (which has a red warning triangle), write a clear description so the AI knows when to call it: Use this tool to search for technical information, logs, or test plans in QA team documents.
- Select your Pinecone credentials and, in the field **Pinecone Index** , choose our common base: n8n-qa-docs.

Schema Table JSON Parameters Settings Execute step

Tip: Get a feel for vector stores in n8n with our [RAG starter template](#)

Credential
Pinecone account

Operation Mode
Retrieve Documents (As Tool for AI Agent)

Description
Folosește această unealtă pentru a căuta informații tehnice, log-uri sau planuri de testare în documentele echipei QA.

Pinecone Index
From list n8n-qa-docs

Limit
4

☒ Include Metadata

☐ Rerank Results

Options
+ Add Option

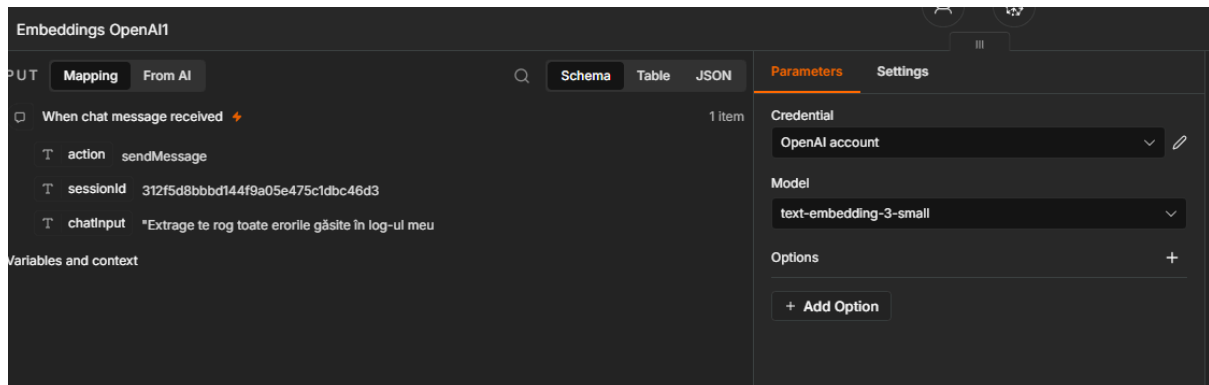
Output
1 item

4. Agent Translator (Embeddings)

For the Agent to search Pinecone, it must "translate" your question from Romanian into the same numbers (Vectors) that we used in Chapter 5.

- Look under your new knot *Pinecone Vector Store* newly attached to the Agent. You will see that it also has a lower port called **Embeddings** (marked with a red star).
- Click on it, search the list and choose **Embeddings OpenAI** (n8n recently grouped them together by putting the word "Embeddings" at the beginning).
- Select your OpenAI credentials and choose the exact same model you used for data

ingestion: text-embedding-3-small.



💡 Architect's Eye (How does RAG work?):

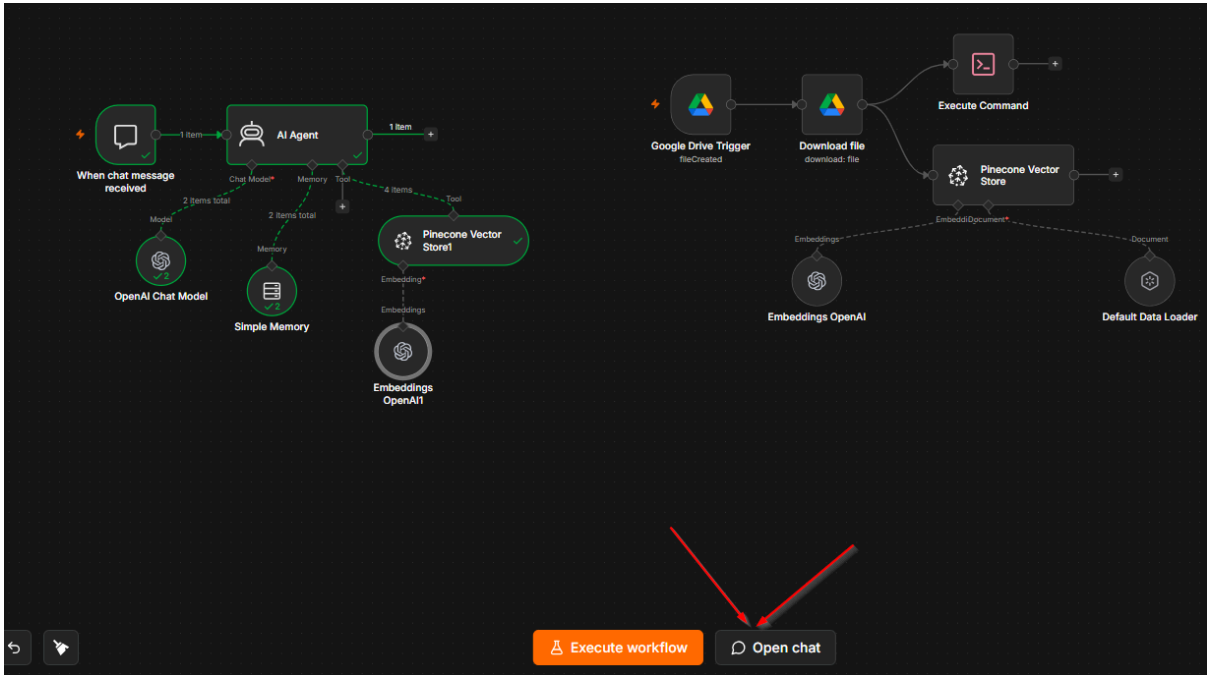
Architecture *Retrieval-Augmented Generation* that you just built works like this:

- You ask: "What are the testing steps in the document?"
- The agent sees that he doesn't know from memory, so he calls the tool *Pinecone*.
- Your question is converted into numbers by the Embeddings node.
- Pinecone quickly searches through the saved fragments in Chapter 5 and returns only the relevant paragraph.
- The agent receives the paragraph, reads it quickly and formulates the perfect answer for you!

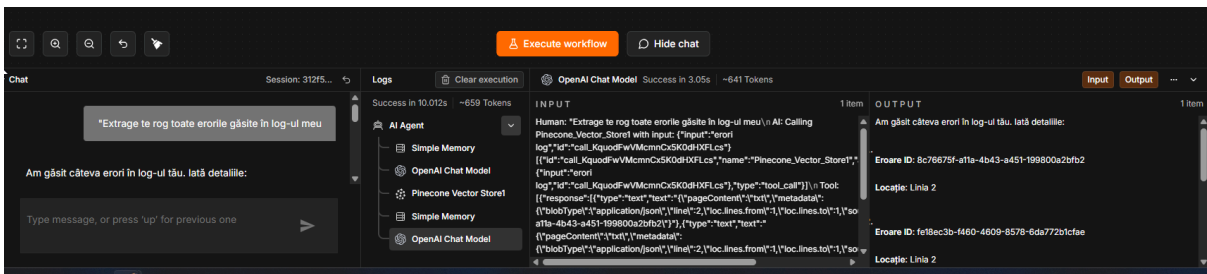
5. The Moment of Truth: The Conversation with Your Data!

Now it's time to enjoy your work. You've created your own personal assistant, based on your local data!

- At the bottom of your screen, you will see a button called **Open chat** (next to the orange Execute button). Press it!



- An interface similar to ChatGPT will open, directly above your board.
- Write a specific question to the assistant related to the content of the test file you downloaded earlier. (Ex: *"Please extract all errors found in my log"*).



Thought Exercise: "The DevOps Personality"

Open the main node window **AI Agent**. You will see a field called **System Message** (or Prompt). The default is empty, and your assistant is polite but generic.

How would you formulate a *System Message* to turn this assistant into a grumpy, life-weary DevOps engineer who gives highly technical but accurate answers using sarcasm? (Try typing your prompt directly into that field and open Chat again!).

